

Question:

Is it the same if I write

$$\frac{dC}{dA} \cdot \frac{dA}{dZ} \cdot \frac{dZ}{db}$$

(how it is written in the lectures and ebook) or

$$\frac{dZ}{db} \cdot \frac{dA}{dZ} \cdot \frac{dC}{dA}$$

(how 3Blue1Brown writes it here and here) when applying the chain rule for $\frac{dC}{db}$?

Answer:

Yes—these are equivalent. Each factor is a number, and multiplication of numbers is commutative, so the product is the same regardless of order. Conceptually, the chain rule just matches dependencies along the path

$$C \rightarrow A \rightarrow Z \rightarrow b$$

. A quick way to see it is if you do the cancellations:

$$\frac{dC}{dA} \cdot \frac{dA}{dZ} \cdot \frac{dZ}{db} = \frac{dC}{db}.$$

Example with fractions

Suppose

$$\frac{dC}{dA} = \frac{1}{2}, \quad \frac{dA}{dZ} = \frac{2}{3}, \quad \frac{dZ}{db} = \frac{3}{5}.$$

Then

$$\frac{dC}{dA} \cdot \frac{dA}{dZ} \cdot \frac{dZ}{db} = \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{3}{5} = \frac{1}{\cancel{2}} \cdot \frac{\cancel{2}}{\cancel{3}} \cdot \frac{3}{5} = \frac{1}{5},$$

and in the reversed order,

$$\frac{dZ}{db} \cdot \frac{dA}{dZ} \cdot \frac{dC}{dA} = \frac{3}{5} \cdot \frac{2}{3} \cdot \frac{1}{2} = \frac{\cancel{3}}{5} \cdot \frac{\cancel{2}}{\cancel{3}} \cdot \frac{1}{\cancel{2}} = \frac{1}{5}.$$

Both give the same result, $\frac{dC}{db} = \frac{1}{5}$.

Working out the math: what does this sequence of chained partial derivatives actually correspond to? This chain of partial derivatives actually corresponds to something. We can calculate each term analytically, fill in the numbers we got from forward propagation (such as the activations in each layer, or the weighted sum we calculated in the network in each layer), and so derive a number for each term. The final number, then, in this case, is the partial derivative of the cost wrt. a certain bias in the network. Here is an illustration, assuming a sigmoid activation function and the mean-squared error as the cost function:

- Single example (one output):

$$C = \frac{1}{2}(y - a^{(L)})^2, \quad a^{(L)} = \sigma(z^{(L)}), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Chain rule (either order):

$$\frac{\partial C}{\partial b^{(L)}} = \frac{\partial C}{\partial a^{(L)}} \cdot \frac{\partial a^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial b^{(L)}} = \underbrace{-(y - a^{(L)})}_{-(y - a^{(L)})} \cdot \underbrace{\sigma'(z^{(L)})}_{\sigma'(z^{(L)})} \cdot \underbrace{1}_1 = (a^{(L)} - y) \sigma'(z^{(L)})(1 - \sigma(z^{(L)})).$$

Reversing the multiplication order gives the same product because these are scalars (i.e. just numbers):

$$\frac{\partial z^{(L)}}{\partial b^{(L)}} \cdot \frac{\partial a^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial C}{\partial a^{(L)}} = 1 \cdot \sigma'(z^{(L)}) \cdot (-(y - a^{(L)})) = (a^{(L)} - y) \sigma'(z^{(L)}).$$

- Dataset of m examples (still one output, now including the **sum terms** for completeness):

$$C = \frac{1}{2m} \sum_{i=1}^m \left(y^{(i)} - a^{(L)}(i) \right)^2, \quad a^{(L)}(i) = \sigma(z^{(L)}(i)).$$

Using linearity of differentiation over the sum, for each example i :

$$\frac{\partial C}{\partial a^{(L)}(i)} = -\frac{1}{m} \left(y^{(i)} - a^{(L)}(i) \right), \quad \frac{\partial a^{(L)}(i)}{\partial z^{(L)}(i)} = \sigma(z^{(L)}(i)) \left(1 - \sigma(z^{(L)}(i)) \right), \quad \frac{\partial z^{(L)}(i)}{\partial b^{(L)}} = 1.$$

Therefore the full derivative (either multiplication order) is

$$\boxed{\frac{\partial C}{\partial b^{(L)}} = \sum_{i=1}^m \frac{\partial C}{\partial a^{(L)}(i)} \cdot \frac{\partial a^{(L)}(i)}{\partial z^{(L)}(i)} \cdot \frac{\partial z^{(L)}(i)}{\partial b^{(L)}} = \frac{1}{m} \sum_{i=1}^m \left(a^{(L)}(i) - y^{(i)} \right) \sigma(z^{(L)}(i)) \left(1 - \sigma(z^{(L)}(i)) \right)}$$

which is identical whether you write the product as:

$$\frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b}$$

or:

$$\frac{\partial z}{\partial b} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial C}{\partial a}$$

Question:

What do you do when optimizing a neural network after you've chained all these partial derivatives together and arrived at the partial derivative of the cost wrt. each parameter?

Answer:

You update all the parameters a little bit using the learning rate to take a small step down the gradient. Remember: like we discussed in the very first lecture when talking about partial derivatives, you treat that big complicated neural net function (it is, in the end, one function, that takes feature inputs and delivers some outputs, like a class prediction) *as if* it depends only on one parameter (say, a specific weight). You then calculate what step you should take to slightly improve that parameter. You do this not for only one weight, but for all weights and biases in the network. Once you know that, you update each of them with a tiny step, by subtracting $\alpha \cdot \text{gradient}$ from the current value:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial C}{\partial \theta}$$

For a specific **weight** connecting neuron k in layer $l-1$ to neuron j in layer l and the **bias** of neuron j in layer l :

$$w_{jk}^{(l)} \leftarrow w_{jk}^{(l)} - \alpha \frac{\partial C}{\partial w_{jk}^{(l)}}, \quad b_j^{(l)} \leftarrow b_j^{(l)} - \alpha \frac{\partial C}{\partial b_j^{(l)}}.$$

To emphasize there are many such updates per neuron and per layer (with ellipses):

$$\begin{aligned}
w_{j1}^{(l)} &\leftarrow w_{j1}^{(l)} - \alpha \frac{\partial C}{\partial w_{j1}^{(l)}} \\
w_{j2}^{(l)} &\leftarrow w_{j2}^{(l)} - \alpha \frac{\partial C}{\partial w_{j2}^{(l)}} \\
&\vdots \\
w_{jk}^{(l)} &\leftarrow w_{jk}^{(l)} - \alpha \frac{\partial C}{\partial w_{jk}^{(l)}} \\
&\vdots \\
b_j^{(l)} &\leftarrow b_j^{(l)} - \alpha \frac{\partial C}{\partial b_j^{(l)}}
\end{aligned}$$

Apply these updates **for all layers and neurons**:

$$\text{for } l = 1, \dots, L, \quad j = 1, \dots, n_l, \quad k = 1, \dots, n_{l-1}.$$

For *one weight*, that looks like one of the small arrows in the image below: one tiny step closer towards an optimum.

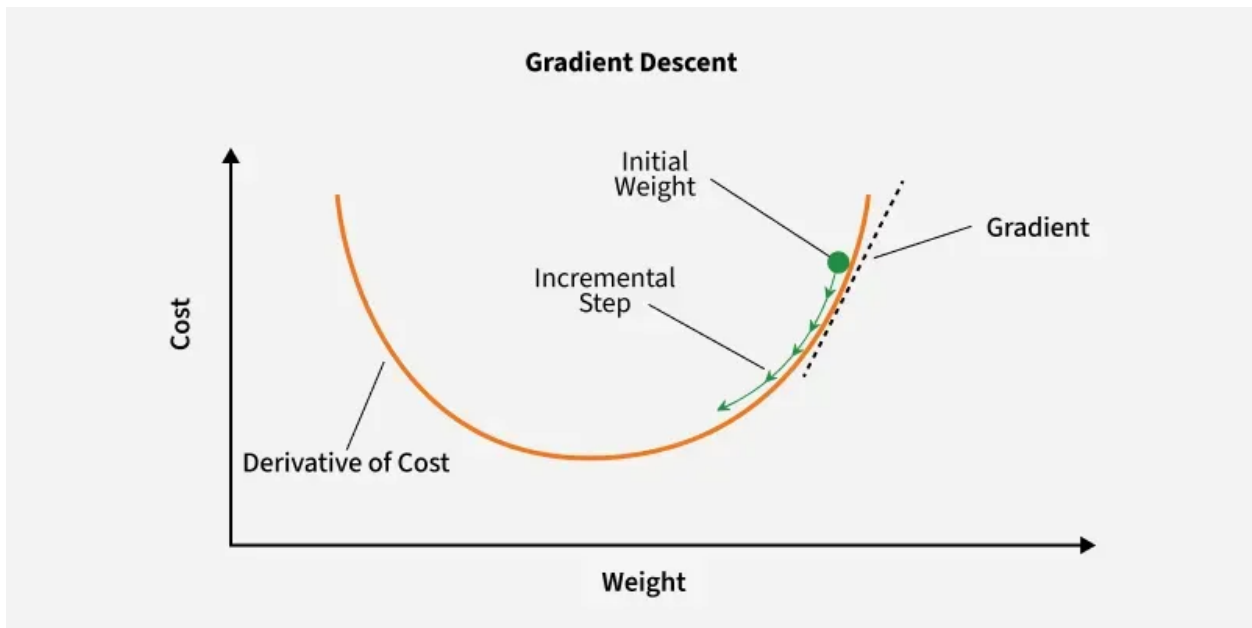


Figure: Visualisation of how the cost function may depend on a certain weight. In reality, we don't know the parabola, we only calculate the gradient at each point, and can take a small step so we move down the gradient, thereby decreasing the cost a little.

However, in reality, you are taking a step on this huge multidimensional cost function surface, using the only tools we have to do so (partial derivatives), hoping that by taking small steps while disregarding that parameters actually affect each other we can still reach a good-enough cost function minimum. See here for a foray into trying to visualise these multidimensional neural network cost surfaces.